

Final Project Report

Team 4 ---- DEPI

PREDICTIVE MAINTENANCE FOR INDUSTRIAL EQUIPMENT

This project focuses on developing a predictive maintenance solution for industrial equipment using machine learning techniques. The goal is to detect early signs of equipment failure, reduce unplanned downtime, optimize maintenance schedules, and improve overall operational efficiency.



TABLE OF CONTENTS

01 Data Collection

Overview of the sensors, readings, and data sources gathered to build the predictive maintenance system.

03 Exploratory Data Analysis (EDA)

Key visual insights showing trends, anomalies, correlations, and patterns within the machine data.

05 Results

Short summary of performance metrics, model comparisons, and the final chosen model outcomes.

02 Data Preprocessing

Summary of cleaning, handling missing timestamps, resampling, interpolation, and preparing the data for modeling.

04 Model Development

Brief overview of the ML and DL models used, training strategy, and the evaluation process.

06 Deployment Flow

High-level description of how the model was integrated into a system for real-time predictions.

07 Challenges Faced During the Project

Summary of major issues such as irregular timestamps, missing data, feature explosion, model instability, and class imbalance.

1. DATA COLLECTION

The Smart Manufacturing IoT-Cloud Monitoring Dataset is designed for real-time process monitoring, predictive maintenance, and anomaly detection in industrial environments. It contains simulated IoT sensor readings collected from manufacturing machines, including parameters such as temperature, vibration, humidity, pressure, and energy consumption.

This dataset is ideal for developing machine learning models aimed at predictive maintenance, helping reduce downtime and optimize operational efficiency. It also includes anomaly flags, failure types, and a target column (`maintenance_required`) indicating whether a machine requires maintenance.

Key Features

- 100,000 sensor records from 50 unique machines
- Timestamps recorded
- Real-time IoT sensor data: Temperature, Vibration, Humidity, Pressure, Energy Consumption
- Machine operational status: Idle, Running, Failure
- Anomaly detection based on extreme sensor readings
- Predictive maintenance labels for ML model training

2. DATA PREPROCESSING

2.1 Cleaning

- Checked duplicates and missing values
- Detected outliers using IQR / Z-score and handled extreme anomalies
- Standardized units and corrected inconsistent formats

2.2 Feature Engineering

- Created aggregated features (rolling averages, rate of change, thermal stress indicators)
- Encoded categorical variables
- Generated interaction terms where correlated patterns existed

2.3 Handling Class Imbalance

Since equipment failures were rare, methods like:

- SMOTE
- Class weights adjustment
- Undersampling majority class
- were explored to improve model learning

3.

EXPLORATORY DATA ANALYSIS (EDA)

3.1 Key Observations

- Certain sensor values (temperature, vibration) showed consistent trends before failures.
- Some machines operated close to thresholds more frequently → higher failure risk.
- Equipment age and workload intensity had strong correlations with failure probability.

3.2 Visualizations

- Distribution plots for sensor readings
- Correlation heatmaps
- Failure rate by machine type or operational condition
- Outlier detection plots

4. MODEL DEVELOPMENT

4.1 Models Tested

- LSTM
- Random Forest
- Gradient Boosting (XGBoost)
- Gradient Boosting (LightGBM)

4.2 Model Training

- Train-test split (80/20)
- Cross-validation
- Hyperparameter tuning (grid search / random search)

4.3 Performance Metrics

- Accuracy
- Precision & Recall
- F1-score (especially important due to class imbalance)
- ROC-AUC
- Confusion matrix

5. RESULTS

5.1 Best Performing Model

Gradient Boosting (or your actual best model—adjust based on your notebook results) delivered:

- Strong generalization across machine types
- Good interpretability via feature importance

5.2 Key Features Identified

- Vibration deviation
- Temperature peaks
- Error code frequency
- Load percentage
- Pressure fluctuations

These contributed heavily to the failure probability.

6. DEPLOYMENT WORKFLOW

6.1 Model Packaging

The LSTM model and scaler are saved (model.pth, scaler.pkl) and loaded for inference. Feature processing and predictions are handled by a single function, enabling portable, consistent deployment.

6.2 Production Integration

Live sensor data + historical readings form sequences for the LSTM to predict Remaining Useful Life (RUL). Alerts are raised if inputs exceed safe historical ranges.

6.3 Monitoring & Updating

Track sensor drift, log predictions, retrain on new failures, and benchmark live performance to maintain reliability.

7. CHALLENGES FACED DURING THE PROJECT

The predictive maintenance project encountered several significant challenges across data quality, feature engineering, and model development. These obstacles directly impacted the performance of the failure classification model and especially the RUL (Remaining Useful Life) regression model. The key challenges are summarized:

7.1 Irregular Timestamp Intervals

The raw sensor readings had inconsistent time gaps:

- Some readings were 5 minutes apart
- Others jumped to 2 hours or more

This inconsistency breaks the assumption of uniform time-series progression.

To correct this, we applied:

- Resampling
- Interpolation

However, this created new problems:

- Large portions of data became missing
- Interpolation introduced artificial patterns
- Real machine behavior was partially lost or distorted

7.2 High Volume of Missing Values After Interpolation

After resampling, gaps in the timeline resulted in:

- Unreliable interpolated values
- Entire sequences becoming unusable
- Dropna() removing many valid rows

This severely reduced usable data quality.

7.3 Using XGBoost to Handle Missing Values

Because traditional ML models dropped too much data, we tested XGBoost, which can handle missing values internally.

We engineered:

- Lag features
- Rolling statistics
- Rate-of-change features

This expanded the dataset to ~78 features, but:

- Performance was still low
- Accuracy and F1-score did not improve
- The model couldn't learn real degradation patterns from noisy data

7.4 Training Separate Models for Each Machine (50 Models)

To capture machine-specific behavior, the dataset was split into 50 individual files, and a model was trained for each machine separately.

Outcome:

No improvement was observed because:

- Each machine still suffered from timestamp irregularities
- Many machines did not have enough data
- Interpolation artifacts remained
- Core dataset issues persisted

This confirmed the problem was global, not machine-specific.

7.5 Weak RUL Signal (Target Variable Issues)

The RUL target

(predicted_remaining_life)

presented its own challenges:

- Weak correlation with input features
- No clear degradation pattern over time
- Sensor values did not gradually change as RUL decreased
- RUL values were highly noisy and difficult to model

Real RUL datasets typically show smooth and predictable degradation trends — this dataset did not.

7.6 Imbalance in Failure Type Classification

The failure type model also suffered from severe class imbalance:

- Some failure classes appeared in less than 2% of the data
- Many classes had too few examples for learning

We tried:

- Oversampling
- Synthetic data generation
- Class weight adjustments

These helped only slightly, but perfect balance was not achievable.

7.8 Data Leakage and Hyped model

- In early experimentation, we built a hybrid model that unintentionally included `machine_status` and `anomaly_flag`, which directly reveal failure conditions.
- This created data leakage, causing the model to achieve extremely high accuracy that did not reflect real predictive performance.
- After removing these features and training an honest, sensor-only model, the results dropped significantly and showed that the dataset had very weak failure patterns.
- This experiment helped us confirm why leakage must be avoided, but the approach was not reliable, produced poor real results, and was not used in the final system.

7.7 Deep Learning Used as the Last Solution

After testing:

- Classical ML
- XGBoost
- Feature-heavy models
- Machine-specific modeling

We finally moved to an LSTM-based deep learning model.

While LSTM improved the performance compared to traditional ML:

- R^2 was still low
- The model only captured limited patterns
- The dataset limitations were too strong to overcome

Deep learning achieved the maximum possible performance given the data quality.

8. FINAL OUTPUTS

8.1 Remaining Useful Life (RUL) Model

This model predicts how many cycles are left before a machine fails, using a time-series LSTM architecture.

Key Steps:

- **Feature Engineering:**
 - 5-step moving averages to smooth noisy sensors
 - Rate-of-change features to capture sudden spikes
 - Selected core sensor inputs (temp, vibration, pressure, etc.)
- **Data Preparation Pipeline:**
 - Oversamples low-RUL samples to handle imbalance
 - Adds light noise to avoid overfitting
 - Standard scaling
 - Converts data into 20-step sequences for LSTM

- **Final Model Architecture:**
 - LSTM with optimized hidden units
 - Dropout for regularization
 - Fully connected output layer → single RUL prediction
- **Training Setup:**
 - Sequence length: 20
 - Momentum: 0.8
 - Threshold: 80
 - Epochs: 50
- **Training Loop:**
 - Evaluates every epoch, updates weights, and automatically saves the best model based on R^2 improvement.

8.2 Machine Health Score Model

This model predicts a Health Score (0–100) that represents the machine's current condition. It uses sensor trends and an optimized LSTM network to detect degradation.

Key Steps:

- **Health Score Creation:**
- Built using sensor Z-scores → combined into a degradation index → scaled to 0–100 and smoothed.
- **Feature Engineering:**
 - 5-step moving averages (temp, vibration)
 - Rate of change features
 - Raw sensor inputs
- **Data Preparation:**
- Standard scaling + 20-step sliding windows for LSTM input.
- **Final Model (Best Trial):**
 - Hidden units: 182
 - Dropout: 0.217
 - Sequence length: 20
 - LR: 0.00054
 - Batch size: 64
 - Loss: MAE
- **Training Results:**
- Strong accuracy with $R^2 \approx 0.90$, stable predictions across the test set.